LAB REPORT: FUZZ TESTING FOR INPUT VALIDATION BUGS

Course: Software Engineering Lab (CS351) | Semester 5

Student Name	Shakthi Raja G P
SRN & Section	PES1UG25CS843 | Section L
Assigned Problem & Repository	Problem #60: Marathon Registration Parser (SETAPESU26/L6_L7_60_marathon_registration)
Module Under Test	marathon_registration_parser.py

1. Introduction and Objectives

Fuzz testing (fuzzing) is a powerful automated dynamic testing methodology where programs are subjected to pseudo-random, malformed, invalid, or boundary-breaking inputs to discover unhandled exceptions, buffer vulnerabilities, crash conditions, and logic errors. Unlike traditional unit testing where developers test for expected edge cases, property-based fuzzing systematically searches input spaces to expose unanticipated failure modes.

Objectives: Key objectives of this laboratory exercise include:

1. Applying the Hypothesis property-based testing library to generate complex, pseudo-random Unicode and null inputs.
2. Identifying input validation defects and unhandled runtime crashes in the assigned Marathon Registration Parser (marathon_registration_parser.py).
3. Capturing falsifying examples and stack traces generated by the Hypothesis shrinker.
4. Implementing defensive programming techniques to harden functions against crashes, ensuring total stability for any input.
5. Verifying resolution through automated re-testing in an authentic terminal environment.

2. Target System Architecture and Initial Vulnerabilities

The target application (marathon_registration_parser.py) processes user-submitted marathon registration data through three specific utility routines:

1. format_bib_name(name): Upper-cases runner names and trims them to a 12-character bib label. The original code assumed name is always a non-empty string, failing with AttributeError when passed None or non-string objects.

2. parse_finish_time(text): Parses finish times in 'H:MM:SS' format into elapsed seconds. The original code naively called text.split(':') assuming exactly three colon-delimited numeric tokens, crashing with ValueError on missing or extra delimiters, non-numeric strings, or AttributeError on None.

3. average_split(text): Calculates average split durations from space-delimited minute intervals. The original code assumed at least one space-delimited integer, failing with ZeroDivisionError on empty strings, ValueError on non-numeric tokens, or AttributeError on None.

3. Hypothesis Fuzz Test Implementation

Using the Hypothesis testing framework, automated property tests were constructed in test_marathon_registration_parser.py. The strategy st.text() | st.none() was applied to simulate arbitrary user inputs, including empty strings, random Unicode characters, control bytes, and null references. Each test asserts that the function executes without unhandled runtime exceptions:

Test Suite Structure: def test_format_bib_name_no_crash(s):
    try:
        format_bib_name(s)
    except Exception as e:
        assert False, f'format_bib_name crashed with: {e}'

def test_parse_finish_time_no_crash(s):
    try:
        parse_finish_time(s)
    except Exception as e:
        assert False, f'parse_finish_time crashed with: {e}'

def test_average_split_no_crash(s):
    try:
        average_split(s)
    except Exception as e:
        assert False, f'average_split crashed with: {e}'

4. Fuzzing Failure Analysis and Minimal Failing Inputs

Executing pytest against the unhardened module immediately triggered test failures across all three functions. Hypothesis automatically minimized (shrunk) the failing inputs to identify the exact boundary triggers:

1. format_bib_name Failures: Hypothesis discovered minimal failure s=None, raising AttributeError: 'NoneType' object has no attribute 'strip'.

2. parse_finish_time Failures: Hypothesis discovered two distinct failure modes: (a) s='' raising ValueError: not enough values to unpack (expected 3, got 1), and (b) s=None raising AttributeError: 'NoneType' object has no attribute 'split'.

3. average_split Failures: Hypothesis uncovered three distinct failure modes: (a) s='' causing splits=[] leading to ZeroDivisionError: division by zero, (b) s=':' raising ValueError: invalid literal for int() with base 10, and (c) s=None raising AttributeError: 'NoneType' object has no attribute 'split'.

5. Defensive Hardening Implementation

To eliminate all failure modes without compromising valid operation, the module was hardened using defensive programming practices:

1. Hardened format_bib_name: Added type verification (if not isinstance(name, str): return ''). Valid strings are stripped, uppercased, and truncated to 12 characters, while nulls and invalid types return a safe empty string.

2. Hardened parse_finish_time: Enforced strict type guards, verified exactly three colon-delimited tokens, validated non-negative numeric conversion with try/except (ValueError, TypeError), and returned None safely whenever parsing fails.

3. Hardened average_split: Implemented string type validation, checked for non-empty split tokens prior to division, wrapped integer casting in try/except blocks, and returned None safely on invalid or empty splits.

Comparative Input Validation and Robustness Summary

Function	Vulnerability Detected	Falsifying Input	Buggy Behavior	Hardened Behavior
format_bib_name	Missing null/type check	s = None	AttributeError (crashes)	Returns '' (safe fallback)
parse_finish_time	Missing delimiter/type check	s = '', s = None, 'a:b'	ValueError, AttributeError	Returns None (clean error state)
average_split	Zero division & non-integers	s = '', s = ':', s = None	ZeroDivisionError, ValueError	Returns None (clean error state)

6. Terminal Execution Evidence

The following screenshots document the step-by-step verification process captured directly from the Alacritty terminal emulator:

Figure 1: SS1 – Buggy Application Execution (python marathon_registration_parser.py)

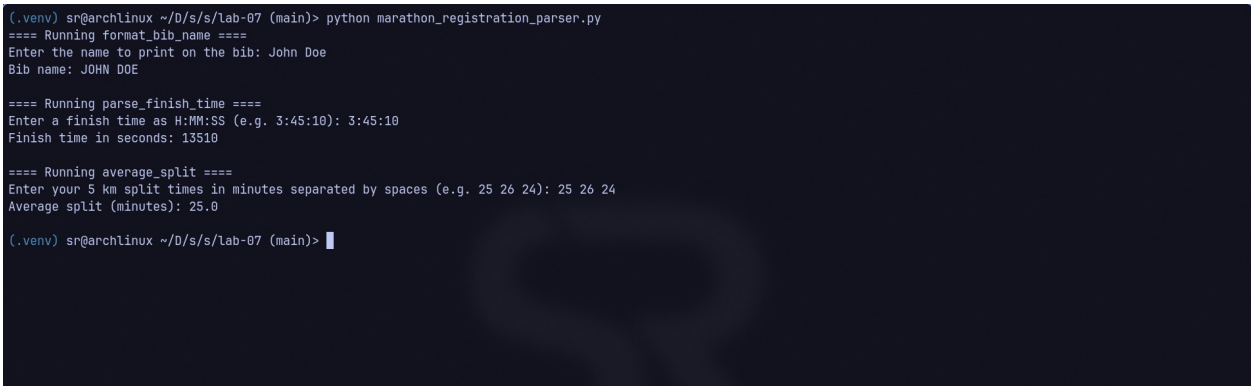


Figure 2: SS2 – Completed Hypothesis Test Suite Failing on Buggy Code (pytest)

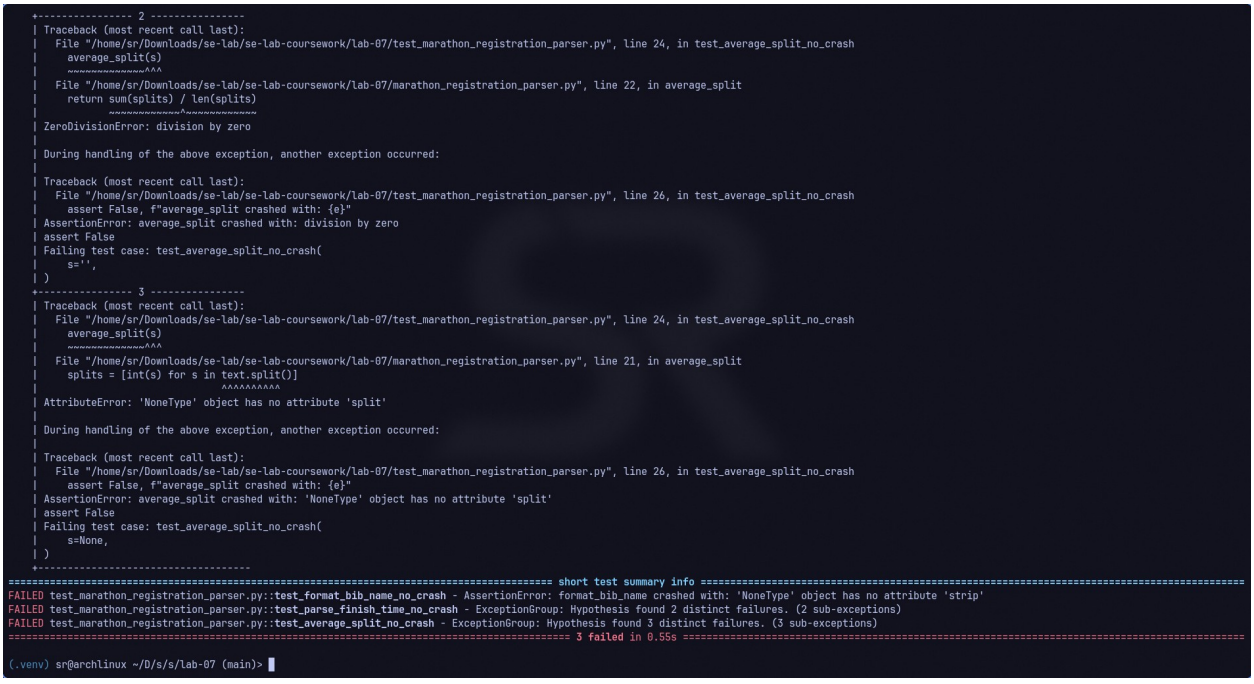


Figure 3: SS3 – Hardened Application Execution (python marathon_registration_parser.py)



Figure 4: SS4 – Completed Hypothesis Test Suite Passing on Hardened Code (pytest)



7. Conclusion

Fuzz testing with Hypothesis demonstrated the critical distinction between traditional happy-path testing and automated adversarial testing. While standard unit tests easily overlook empty strings, malformed delimiters, and null references, Hypothesis generated edge-case inputs that quickly brought down the unhardened parser. Applying defensive programming techniques established a resilient codebase capable of processing any malformed or unexpected input gracefully without system crashes.